

**CSE-4155 Introduction to Machine Learning Lab**  
**4th Year 1st Semester, 2026**  
**Department of Computer Science and Engineering, University of Dhaka**  
**Course Instructor: Dr. Muhammad Ibrahim**

**Lab 03 (04-10-2026)**

The students are required to bring their codes on the following (for binary classification only):

- Interacting with decision boundary
- Perceptron: code from scratch.

**1. Decision Boundary Investigation**

Load the data from the attached *perceptron\_data.csv* file into your code. The data file has two features and a target variable.

Assume a decision boundary with random weights,  $w_0, w_1, w_2$

Plot the resulting decision boundary:  $w_0 + w_1x_1 + w_2x_2 = 0$  together with the training data.

Change the values of  $w_0, w_1, w_2$  manually and observe how the decision boundary moves.

**2. Implementation of a Perceptron classifier without using a machine-learning classification library.**

Your program should contain separate functions for:

```
initialize_parameters()
predict()
compute_loss()
update_weights()
train_perceptron()
```

For every training example:

1. Calculate the prediction.
2. Compare it with the actual label.
3. If the example is misclassified, update the parameters.
4. Continue for a specified number of epochs.

Use the Perceptron update rule:

$$w \leftarrow w + \eta y^{(i)} x^{(i)} \text{ for all } j, \text{ when } y^{(i)}(w^\top x^{(i)}) \leq 0 \text{ for a misclassified example } x^{(i)}.$$

Record the number of misclassified examples after every epoch.

### 3. Visualize Perceptron Learning

Modify your program so that it:

1. Plots the training data.
2. Plots the initial decision boundary.
3. Trains the perceptron.
4. Plots the final decision boundary.
5. Plots the number of classification errors against epoch numbers.

Then understand:

- How did the decision boundary change during training?
- After how many epochs did the model stop making classification errors?
- Did the final model correctly classify every training example?

### 4. Investigate a Limitation of the Perceptron

Below is a small dataset representing the **XOR problem**:

| x1 | x2 | Label |
|----|----|-------|
| 0  | 0  | +1    |
| 0  | 1  | -1    |
| 1  | 0  | -1    |
| 1  | 1  | +1    |

Apply your perceptron implementation to this dataset and report what happens.

### Regarding Deliverables

- 1) Do not upload any zip file.
- 2) Submit the code file(s) only. No data files need to be uploaded.
- 3) Filenames should have format: `<roll>_<name>.<ext>`.

Example: `08_Rokonuzzaman.ipynb`

You shall not be allowed to submit your work after the lab hours, so make sure you prepare your deliverables in a timely manner.

**IMPORTANT:** You will be evaluated on the cleanliness and modularity of your code. This is the framework that you'll use throughout the entire course, and eventually for your lab project. So make it as standard as possible.